Run Existing MCP Servers

Estimated time needed: **30** minutes

MCP servers will do to LLMs what the app store did to mobile phones. Imagine doing things that you do online with an LLM - with MCP servers you can get your LLM to book a hotel or a flight and much more. Here we will use an MCP server to get coding information for your LLM. In this project you will be introduced to using MCP servers, using transport protocols as a bridge from your application to the MCP server. We will introduce two transport protocols; the STDIO protocol, which will allow you to run MCP servers locally on your machine, and the HTTP protocol which will allow you to work with MCP servers hosted anywhere with internet access, as shown in the flowchart below.

```
Client or Application  ←→  [transport]  ←→  MCP Server
     (LLM)           (Bridge)        (Local/Remote)
```

Once you get the connection, the process is identical.

Objectives

After completing this lab you will be able to:

- Create **STDIO** and **HTTP** transports using the FastMCP library to configure connections between MCP clients and servers.
- Perform asynchronous requests to the MCP server to list and call tools.
- Call tools from the Context7 MCP Server (library/framework documentation formatted for LLMs) via STDIO and HTTP.

Setup

For this lab, we will only need the following library:

- `fastmcp` for creating and connecting MCP servers, clients, and hosts

Installing Required Libraries

Run the following cell to install the `fastmcp` library:

In [1]:
```
%%capture
%pip install fastmcp
```

STDIO Review

STDIO or Standard Input/Output is three communication channels that every program automatically gets. This was originally designed for Linux/Unix systems but is now used across all operating systems. This is for MCP servers that run locally.

STDOUT

`STDOUT` (Standard Output) – The default stream where a program writes its normal output (usually your screen/terminal). We can print output using `print()`

In [2]:
```
my_code="hello"
print(my_code)
```

```
hello
```

We can sketch the process.

My Python Program: my_code = "hello" → print() →` [STDOUT] → Screen shows: hello

`sys.stdout.write()` is a low-level (primitive) function that writes raw text directly to the standard output stream.

In [2]:
```
import sys
sys.stdout.write("Hello")
```

```
Hello
```

Out[2]:
```
5
```

The 5 is just Jupyter displaying the return value

STDIN

`STDIN` (Standard Input) - Where input comes FROM (like your keyboard). Here we call the function input() with the prompt "What is your name?" You will then be prompted to input your name. The input from your keyboard will be stored in the variable name.

In [3]:
```
name = input("What is your name?")

sys.exit("Notebook exited after input.")
```

```
An exception has occurred, use %tb to see the full traceback.

SystemExit: Notebook exited after input.
```

```
/opt/conda/lib/python3.12/site-packages/IPython/core/interactiveshell.py:358
5: UserWarning: To exit: use 'exit', 'quit', or Ctrl-D.
  warn("To exit: use 'exit', 'quit', or Ctrl-D.", stacklevel=1)
```

We can sketch the process. We see that there are several steps:

```
My Python Program → [STDOUT] → You (screen)
      input()      "What is your name?"
         ↓
You (keyboard) → [STDIN] → My Python Program
    "Alice"                    stores in 'name'
         ↓
My Python Program → [STDERR] → You (screen)
     sys.exit()    "Notebook exited after input."
```

We can see the varable `name` has the input value.

In [4]:
```
print(name)
```

```
raj
```

STDERR

`STDERR` (standard error) is reserved for error messages and diagnostics. Both usually appear on the screen, but since they are separate streams, you can redirect them independently (e.g., send results to a file while keeping errors visible).

In [5]:
```
# Normal output goes to stdout
print("This is standard output (stdout)")

# Error messages can be written to stderr
print("This is an error message (stderr)", file=sys.stderr)

# Example: catching a real error
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error occurred: {e}", file=sys.stderr)
```

```
This is standard output (stdout)
This is an error message (stderr)
Error occurred: division by zero
```

Importing Required Libraries

Run the following cell to import all required modules and dependencies for this lab. We'll need:

- `Client`: A communication outlet between an application (host) and an MCP server.

- `StdioTransport`: Communication run on subprocesses (read only input, write only output) for local MCP servers.
- `StreamableHttpTransport`: Communication for production deployments that run over bidirectional streaming on HTTP connections.

In [6]:
```
from fastmcp import Client
from fastmcp.client.transports import StreamableHttpTransport, StdioTranspor
```

Context7

In this lab we'll connect to the Context7 MCP server via two different transport methods (STDIO and HTTP). Context7 is a service (with an MCP server) that provides up-to-date code documentation for LLMs and AI code editors. Once connected, we'll create `Client` instances, list the available tools, and call on those tools to get library documentation.

Input (what you give it):

- Library names (such as "fastmcp")
- Search queries for code documentation
- Requests for specific library information

Output (what it gives you back):

- Up-to-date code documentation formatted for LLMs
- Library information and compatibility details
- Code examples and usage patterns
- API references and function descriptions

STDIO Transport

For `StdioTransport`, everything runs on your system. STDIO Transport acts like a bridge from your Python file to an MCP server. Your notebook/Python file runs the `StdioTransport` code, your computer downloads the Context7 server code via npx, and your computer starts the Context7 server as a separate process. Both programs (your notebook + Context7 server) run on your machine, with STDIO Transport serving as the bridge (or wire) that connects your Python client to the Context7 server. This is the simplest way to connect to a local or CLI-packaged server as shown in the following diagram.

```
Your Python Code  ←→  [stdio_transport]  ←→  Context7 Server
                         on you com
    (Client)              (Bridge)              (your system)
```

Let's create the transport.

The `StdioTransport`:

- **Launches** the Context7 server as a subprocess (`npx @upstash/context7-mcp`)

npx - A Node.js

- Downloads the Context7 MCP server code from the npm registry
- Installs it temporarily (with -y flag to auto-confirm)
- Runs the MCP server program
- Starts listening on STDIN/STDOUT for MCP protocol messages

Python equivalent analogy: bash#

Similar to doing this in Python world:

- pip install context7-mcp-server
- python -m context7-mcp-server

Why use npx?

- **Connects** your Python code's messages to the server's STDIN
- **Connects** the server's STDOUT back to your Python code
- **Manages** the communication flow between both sides

In [7]:
```
stdio_transport = StdioTransport(
    command = "npx",
    args=["-y", "@upstash/context7-mcp"]
)
print(stdio_transport )
```

```
<StdioTransport(command='npx', args=['-y', '@upstash/context7-mcp'])>
```

We'll instantiate an `StdioTransport` with the following parameters:

- `command="npx"` : We use `npx` to run the published Context7 MCP server package
- `args=[...]` : The **Node** package that contains the server with the `-y` flag to say "yes" to all prompts during installation
- Think of this as: “Start the server for me and wire up stdin/stdout so we can chat.”

Client

Now let's wrap the transport in a client. Since the client needs a `stdio_transport` object, we defined the transport 'Client':

In [8]:
```
stdio_client = Client(stdio_transport)
```

The `Client` gives you a simple way to access the MCP server. Because communication with a server can take some time, Python uses asynchronous methods `async/await` to prevent the program from freezing while it waits for a response.

The line `async with stdio_client as client` uses Python's asynchronous context manager. This means that `Client` runs inside a special block that automatically takes care of starting and closing the connection, similar to how `with open(...)` works for files.

In this example, the method `await client.list_tools()` asks the MCP server for a list of available tools and stores them in the `tools` object. The `await` keyword tells Python to pause this task until the server responds, while still allowing other tasks to run in the meantime. `await` can only be called in an asynchronous function or process.

In [9]:
```
async with stdio_client as client:
    # List of tools the server provides
    tools = await client.list_tools()

print("Done")
```

```
Done
```

In [10]:
```
len(tools)
```

Out[10]:
```
2
```

The tools list contains a set of tool objects - in this case, two tools. We can get the name of the first tool by accessing `tools[0].name`, which returns "resolve-library-id".

In [21]:
```
print(tools[1].name)
```

```
query-docs
```

`resolve-library-id` is a search tool that helps you find the correct library identifier in Context7's system. When you search for a library name such as "pandas" or "requests", this tool finds the specific Context7-compatible ID (like "/pandas-dev/pandas") that you need to retrieve documentation.

We can also access the tool's description, which provides detailed information about what the tool does and how to use it.

In [22]:
```
print(tools[1].description)
```

```
Retrieves and queries up-to-date documentation and code examples from Contex
t7 for any programming library or framework.

You must call 'Resolve Context7 Library ID' tool first to obtain the exact C
ontext7-compatible library ID required to use this tool, UNLESS the user exp
licitly provides a library ID in the format '/org/project' or '/org/project/
version' in their query.

IMPORTANT: Do not call this tool more than 3 times per question. If you cann
ot find what you need after 3 calls, use the best information you have.
```

The `inputSchema` defines that this tool requires a `libraryName` parameter as a string input, this is where you provide the library name you're searching for.

In [13]:
```
tools[0].inputSchema
```

Out[13]:
```
{'$schema': 'http://json-schema.org/draft-07/schema#',
 'type': 'object',
 'properties': {'query': {'type': 'string',
   'description': 'The question or task you need help with. This is used to
rank library results by relevance to what the user is trying to accomplish.
The query is sent to the Context7 API for processing. Do not include any se
nsitive or confidential information such as API keys, passwords, credential
s, personal data, or proprietary code in your query.'},
  'libraryName': {'type': 'string',
   'description': 'Library name to search for and retrieve a Context7-compa
tible library ID.'}},
 'required': ['query', 'libraryName']}
```

The second tool can be accessed using `tools[1]`. We can examine its details with `tools[1].name`, `tools[1].description`, and `tools[1].inputSchema`:

In [14]:
```
print(
f""" name: {tools[1].name}: \n
description: {tools[1].description} \n
inputSchema: {tools[1].inputSchema}""")
```

```
 name: query-docs:

description: Retrieves and queries up-to-date documentation and code example
s from Context7 for any programming library or framework.

You must call 'Resolve Context7 Library ID' tool first to obtain the exact C
ontext7-compatible library ID required to use this tool, UNLESS the user exp
licitly provides a library ID in the format '/org/project' or '/org/project/
version' in their query.

IMPORTANT: Do not call this tool more than 3 times per question. If you cann
ot find what you need after 3 calls, use the best information you have.

inputSchema: {'$schema': 'http://json-schema.org/draft-07/schema#', 'type':
'object', 'properties': {'libraryId': {'type': 'string', 'description': "Exa
ct Context7-compatible library ID (e.g., '/mongodb/docs', '/vercel/next.js',
'/supabase/supabase', '/vercel/next.js/v14.3.0-canary.87') retrieved from 'r
esolve-library-id' or directly from user query in the format '/org/project'
or '/org/project/version'."}, 'query': {'type': 'string', 'description': "Th
e question or task you need help with. Be specific and include relevant deta
ils. Good: 'How to set up authentication with JWT in Express.js' or 'React u
seEffect cleanup function examples'. Bad: 'auth' or 'hooks'. The query is se
nt to the Context7 API for processing. Do not include any sensitive or confi
dential information such as API keys, passwords, credentials, personal data,
or proprietary code in your query."}}, 'required': ['libraryId', 'query']}
```

The second tool `query-docs` actually downloads the documentation for a library. You must first use the `resolve-library-id` tool to get the correct library ID (like "/pandas-dev/pandas"), then use this second tool with that ID to retrieve it.

Let's use the context manager again to call the `resolve-library-id` tool and search for `fastmcp` documentation.

call_tool

`call_tool` is the method you use to actually run/execute a specific tool on the MCP server.

The pramaters are:

- **`tool_name`** - The tool name comes from `.name` like `tools[0].name`, which gives you "resolve-library-id"
- **`{parameters}`** - The inputs the tool needs (like `{"libraryName": "fastmcp", "query":"[YOUR QUERY]"}`). The input structure comes from `.inputSchema`
- **`response`** - What the tool sends back to you

resolve-library-id

Let's get information about the FastMCP library. We need the library identification, so we'll use the `resolve-library-id` tool. This will allow us to get the library information. The input parameter will be libraryName: "fastmcp".

In [15]:
```
async with stdio_client as client:
    # Find a library ID via a search query
    response = await client.call_tool("resolve-library-id", {
        "libraryName": "fastmcp",
        "query": "I want to create a new MCP server using the fastmcp Pythor
    })

print(response.content[0].text)
```

```
Available Libraries:

- Title: FastMCP
- Context7-compatible library ID: /jlowin/fastmcp
- Description: FastMCP is a Python framework for building Model Context Prot
ocol (MCP) servers and clients, simplifying the creation of LLM-integrated a
pplications with features like deployment, authentication, and dynamic tool
generation.
- Code Snippets: 347
- Source Reputation: High
- Benchmark Score: 71.71
- Versions: v3.0.0b1, v2.14.4, v3.0.0b2, v2.14.5, v3.0.0rc2, v3.0.1, v3.0.2,
v3.1.0
----------
- Title: FastMCP
- Context7-compatible library ID: /llmstxt/gofastmcp_llms_txt
- Description: FastMCP is a fast, Pythonic framework for building and deploy
ing Multi-modal Conversational Platform (MCP) servers and clients, offering
programmatic interfaces and extensive integrations for LLMs and various serv
ices.
- Code Snippets: 4366
- Source Reputation: High
- Benchmark Score: 76.22
----------
- Title: FastMCP
- Context7-compatible library ID: /websites/gofastmcp
- Description: FastMCP is a Python framework for building Model Context Prot
ocol (MCP) servers and clients that connect LLMs to tools and data in a prod
uction-ready, Pythonic way.
- Code Snippets: 595
- Source Reputation: High
- Benchmark Score: 49.22
----------
- Title: FastMCP
- Context7-compatible library ID: /llmstxt/gofastmcp_llms-full_txt
- Description: FastMCP is a Python framework for building MCP servers, featu
ring an experimental OpenAPI parser, JWT claims, and optimized payload handl
ing.
- Code Snippets: 12294
- Source Reputation: High
- Benchmark Score: 65.84
----------
- Title: FastMCP
- Context7-compatible library ID: /websites/gofastmcp_v2_getting-started
- Description: FastMCP is a Python framework for building Model Context Prot
ocol (MCP) servers and clients with a clean, Pythonic interface, providing p
roduction-ready features including enterprise authentication, deployment too
ls, and comprehensive client libraries.
- Code Snippets: 36
- Source Reputation: High
- Benchmark Score: 62.29
```

When you call the `resolve-library-id` tool, it returns a list of matching libraries for your search term. The output shows multiple options with details to help you choose the best one. Each result includes a Library ID (the unique identifier you'll need for the next

step), a description of what the library does, the number of code snippets available, and a benchmark score indicating reliability.

For example, we will select the following result:

```
- Title: FastMCP
- Context7-compatible library ID: /llmstxt/gofastmcp_llms-
full_txt
- Description: FastMCP is a Python framework for building MCP
servers, featuring an experimental OpenAPI parser, JWT
claims, and optimized payload handling.
- Code Snippets: 12289
- Source Reputation: High
- Benchmark Score: 79
```

Taking a look at the list, we'll choose the library with the highest **Benchmark Score**. The **Context7-compatible library ID** `/llmstxt/gofastmcp_llms-full_txt` has a Trust Score of 9.6 so we'll go with that one.

query-docs

Now we'll use the `query-docs` tool to retrieve the actual documentation for that specific library.

We'll use the context manager once again to fetch the code snippets and documentation of `/llmstxt/gofastmcp_llms-full_txt` using the `query-docs` tool.

In [23]:
```
async with stdio_client as client:
    # Use resolved ID to fetch documentation
    docs = await client.call_tool("query-docs", {
        "libraryId": "/llmstxt/gofastmcp_llms-full_txt",
        "query": "I want to fetch the code snippets and the documentation",
        "tokens": 5000
    })

    print(docs.content[0].text[:1000])
```

```
### Fetch All Tools Automatically (JavaScript)

Source: https://gofastmcp.com/servers/pagination

This snippet demonstrates how to fetch all available tools using the Gofastm
cp client. It automatically handles pagination to retrieve all 200 tools. Th
e output includes the total number of tools fetched.

```javascript
import { Client } from "fastmcp"

const client = new Client()

// Returns all 200 tools, fetching pages automatically
const tools = await client.list_tools()
print(f"Total tools: {len(tools)}")
```

Client Fetches All Paginated Items Automatically

Source: https://gofastmcp.com/servers/pagination

Demonstrates how the FastMCP Client transparently handles pagination. Conven
ience methods like `list_tools()` automatically fetch all pages and return t
he complete list, ensuring existing code functions without modification.

```python
from fastmcp import Client

async with Client(server) as client:
    # Returns all 200 tools, fetching pages
```
```

As you can see, we have printed out the first 1000 characters of the library documentation of `/punkpeye/fastmcp`. This documentation is what LLMs and AI code agents use to keep their knowledge on common libraries and frameworks up-to-date.

Question 1. How do you use the resolve-library-id tool to find the library ID for scikit-learn

In [28]:
```python
async with stdio_client as client:
   # Find a library ID via a search query
   response = await client.call_tool("resolve-library-id", {
       "libraryName": "scikit-learn",
       "query": "I want to use scikit-learn package"
   })

   print(response.content[0].text[:1500])
```

```
Available Libraries:

- Title: Scikit-learn
- Context7-compatible library ID: /scikit-learn/scikit-learn
- Description: scikit-learn: machine learning in Python
- Code Snippets: 3274
- Source Reputation: High
- Benchmark Score: 80.45
- Versions: 1.7.1
----------
- Title: scikit-learn
- Context7-compatible library ID: /websites/scikit-learn_stable
- Description: scikit-learn is an open-source Python library providing simpl
e and efficient tools for predictive data analysis, including algorithms for
classification, regression, clustering, dimensionality reduction, and model
selection.
- Code Snippets: 28571
- Source Reputation: High
- Benchmark Score: 75.03
----------
- Title: scikit-learn
- Context7-compatible library ID: /websites/scikit-learn_dev
- Description: scikit-learn is a free and open-source Python machine learnin
g library that provides simple and efficient tools for data analysis, predic
tive modeling, and statistical learning.
- Code Snippets: 29216
- Source Reputation: High
- Benchmark Score: 65.2
----------
- Title: Extension for Scikit-learn
- Context7-compatible library ID: /uxlfoundation/scikit-learn-intelex
- Description: Extension for Scikit-learn is a free software AI accelerator
designed to deliver 10-100x performance acceleration to existing scikit-lear
n applications on CPUs and GPUs across single- and multi-node configuration
s, often without code modifications.
- Code Snippets: 697
- Source Reputation: High
- Benchmark Score: 69.08
----------
- Title: scik
```

▶ Click here for hint

Question 2. How do you get the actual documentation once you have the library ID?

In [29]:
```
async with stdio_client as client:
    # Use resolved ID to fetch documentation
    docs = await client.call_tool("query-docs", {
        "libraryId": "scikit-learn/scikit-learn",
        "query": "I want to fetch the code snippets and the documentation",
        "tokens": 5000
    })
```

```
    print(docs.content[0].text[:1000])
```

```
### Format Code Snippets and Error Messages

Source: https://github.com/scikit-learn/scikit-learn/blob/main/doc/developer
s/tips.rst

Demonstrates how to use Markdown code blocks for Python code and traceback m
essages to improve readability in issue reports and comments. Proper formatt
ing aids maintainers in understanding and reproducing problems.

```python
```python
print(something)
```
```


```python
```pytb
Traceback (most recent call last):
    File "<stdin>", line 1, in <module>
ImportError: No module named 'hello'
```
```


--------------------------------

### Fetch 20 newsgroups dataset (Python)

Source: https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/datas
ets/descr/twenty_newsgroups.rst

Fetches the 20 newsgroups dataset, returning raw text data. It can download
data from the original website, extract it, and load specified subsets (trai
n/test) or categories. The output includes filenames, target labels, and cat
egory names.

```python
from sklearn.datasets import
```

▶ Click here for hint

HTTP Preface

The HTTP protocol allows data to be transferred over the internet. Since the protocol can be somewhat complex, we will focus on the GET method, one of the most common HTTP methods.

The figure below illustrates a typical response. The response start line contains:

- Version number (e.g., HTTP/1.0)
- Status code (e.g., 200 for success)
```

- Descriptive phrase (e.g., OK)

The response header follows, providing useful metadata. Finally, the response body includes the requested resource, such as an HTML document. It should also be noted that some requests include headers, which provide additional details or parameters for the transaction.

Requests is a Python Library that allows you to send `HTTP/1.1` requests easily. We can import the library as follows:

In [30]:
```
import requests
```

You can make a `GET` request via the method `get` to www.ibm.com:

In [31]:
```
url='https://www.ibm.com/'
r=requests.get(url)
```

We have the response object `r`, this has information about the request, such as the status of the request. We can view the status code using the attribute `status_code`.

In [32]:
```
r.status_code
```

Out[32]:
```
200
```

You can view the request headers and the body (there is no body for a get request so we get 'None'):

In [33]:
```
print(r.request.headers)
print("request body:", r.request.body)
```

```
{'User-Agent': 'python-requests/2.32.3', 'Accept-Encoding': 'gzip, deflate,
br, zstd', 'Accept': '*/*', 'Connection': 'keep-alive', 'Cookie': '_abck=82A
70D3EDECB737F819C477E8DC85309~-1~YAAQqonMF8JlKbicAQAAD4fjyw9hoOkf7QUuv0XONIA
lC0CcMtF4Cu/IyeJRnTDG0LNA9rVE8Bbx4VYyXyjyPQpJhPVzU0e6TnxNPH/mKuTnLBHzCiw2qoe
pcL7l7dHoq8RBjt6jKxKVDXhoiqKAi4wEgrvHiJdbyLL8U4160WfWuo+bQ+VXFouWozz7EGm/1gB
Q1wmbkUFe/Tl62EhNpWovWE9pUmWdVNZOtgGDGES5ViRqThTC4pm693GXqjWpJc/lTDFmGp1l+Z8
Cc1In/x4TO2JGJlLNOh9vJDe+PlKv7/a4jGFTDncaUPlWRPXpqCyt8DtiIObva9p0zUNOUzOer6S
TnlGIYMH1wYZzNji6RnDc9xrrm6XKg6FPBTTBR62w7hNndVcGKljpfTlAUyZDxppozKAYGi37MF0
qrL0Fg8OcIA0tHxgdE6NcWMmCVI0=~-1~-1~-1~-1~-1; bm_sz=51D4CDFD3F070249C966D1C4
F29FB027~YAAQqonMF8NlKbicAQAAD4fjyx+Bn0yDw5Q3duaopp4AJXfl4lPksgfk0ooE2WBiHW8
yH4xdS9lL1maAJ2InUtpE3nhfTzSbGc3D2deoEF/4TE++sIqny/KiMbvZ0VdLvdyUs9deCWCUawm
tSUxouRGN2SPOV3tVwvtSAWWx3p5nPqLB/buT9ZZJKXxyPd1fWdqeJFehJgl36W56pInvivWHFmm
qTtkEtZ/gfhwJ6LXiTkKDG7wXbarU8YBUslOKoALzLJpJS49JeQVc6b0ow2AfF5jMEv00F00INdr
BlrQux01dxuVUcRTfDgfMpSzb3QK3hUcdB2Peuba4Fa7wpr1vaM1pHcjbTgSZ~3622456~440839
0'}
request body: None
```

HTTP Transport

Now let's talk to the Context7 MCP server over HTTPS—no local process needed. The HTTP transport also acts like a bridge, but instead of connecting from your notebook/Python file to your local computer, it connects to a remote server. This is ideal when the server is hosted remotely or you don't want to manage a subprocess. We'll use the same Client API with a different "wire" (transport). Once you create the transport, the process is pretty much identical.

```
Your Python Code  ←→  [http_transport]  ←→  Context7 Server
                         (remote)
    (Client)             (Bridge)              (Context7's
                         servers)
```

First, we create the `StreamableHttpTransport` object.

In [34]:
```
http_transport = StreamableHttpTransport(
    url="https://mcp.context7.com/mcp"
)
```

- `url`: The server's MCP endpoint that exposes all the tools, prompts, and resources
- “Streamable” means responses can arrive incrementally; Client handles this for you.

Let's wrap the transport in a Client.

In [35]:
```
http_client = Client(http_transport)
```

Finally, like before, let's list and call the tools with the same configuration as above.

In [36]:
```
async with http_client as client:
    tools = await client.list_tools()

    response = await client.call_tool("resolve-library-id", {
        "libraryName": "fastmcp",
        "query": "I want to create a new MCP server using the fastmcp Pythor
    })

    docs = await client.call_tool("query-docs", {
        "libraryId": "/llmstxt/gofastmcp_llms-full_txt",
        "query": "I want to fetch the code snippets and the documentation",
        "tokens": 5000
    })
```

Let's check by printing out the outputs, they should match to the outputs above.

In [37]:
```
for tool in tools:
        print(
f"""{tool.name}: \n
{tool.description} \n
{tool.inputSchema}""")
```

```
print(response.content[0].text[:1000])
print(docs.content[0].text[:500])
```

```
resolve-library-id:

Resolves a package/product name to a Context7-compatible library ID and retu
rns matching libraries.

You MUST call this function before 'Query Documentation' tool to obtain a va
lid Context7-compatible library ID UNLESS the user explicitly provides a lib
rary ID in the format '/org/project' or '/org/project/version' in their quer
y.

Each result includes:
- Library ID: Context7-compatible identifier (format: /org/project)
- Name: Library or package name
- Description: Short summary
- Code Snippets: Number of available code examples
- Source Reputation: Authority indicator (High, Medium, Low, or Unknown)
- Benchmark Score: Quality indicator (100 is the highest score)
- Versions: List of versions if available. Use one of those versions if the
user provides a version in their query. The format of the version is /org/pr
oject/version.

For best results, select libraries based on name match, source reputation, s
nippet coverage, benchmark score, and relevance to your use case.

Selection Process:
1. Analyze the query to understand what library/package the user is looking
for
2. Return the most relevant match based on:
- Name similarity to the query (exact matches prioritized)
- Description relevance to the query's intent
- Documentation coverage (prioritize libraries with higher Code Snippet coun
ts)
- Source reputation (consider libraries with High or Medium reputation more
authoritative)
- Benchmark Score: Quality indicator (100 is the highest score)

Response Format:
- Return the selected library ID in a clearly marked section
- Provide a brief explanation for why this library was chosen
- If multiple good matches exist, acknowledge this but proceed with the most
relevant one
- If no good matches exist, clearly state this and suggest query refinements

For ambiguous queries, request clarification before proceeding with a best-g
uess match.

IMPORTANT: Do not call this tool more than 3 times per question. If you cann
ot find what you need after 3 calls, use the best result you have.

{'$schema': 'http://json-schema.org/draft-07/schema#', 'type': 'object', 'pr
operties': {'query': {'type': 'string', 'description': 'The question or task
you need help with. This is used to rank library results by relevance to wha
t the user is trying to accomplish. The query is sent to the Context7 API fo
r processing. Do not include any sensitive or confidential information such
as API keys, passwords, credentials, personal data, or proprietary code in y
our query.'}, 'libraryName': {'type': 'string', 'description': 'Library name
```

```
to search for and retrieve a Context7-compatible library ID.'}}, 'required':
['query', 'libraryName']}
query-docs:

Retrieves and queries up-to-date documentation and code examples from Contex
t7 for any programming library or framework.

You must call 'Resolve Context7 Library ID' tool first to obtain the exact C
ontext7-compatible library ID required to use this tool, UNLESS the user exp
licitly provides a library ID in the format '/org/project' or '/org/project/
version' in their query.

IMPORTANT: Do not call this tool more than 3 times per question. If you cann
ot find what you need after 3 calls, use the best information you have.

{'$schema': 'http://json-schema.org/draft-07/schema#', 'type': 'object', 'pr
operties': {'libraryId': {'type': 'string', 'description': "Exact Context7-c
ompatible library ID (e.g., '/mongodb/docs', '/vercel/next.js', '/supabase/s
upabase', '/vercel/next.js/v14.3.0-canary.87') retrieved from 'resolve-libra
ry-id' or directly from user query in the format '/org/project' or '/org/pro
ject/version'."}, 'query': {'type': 'string', 'description': "The question o
r task you need help with. Be specific and include relevant details. Good:
'How to set up authentication with JWT in Express.js' or 'React useEffect cl
eanup function examples'. Bad: 'auth' or 'hooks'. The query is sent to the C
ontext7 API for processing. Do not include any sensitive or confidential inf
ormation such as API keys, passwords, credentials, personal data, or proprie
tary code in your query."}}, 'required': ['libraryId', 'query']}
Available Libraries:

- Title: FastMCP
- Context7-compatible library ID: /jlowin/fastmcp
- Description: FastMCP is a Python framework for building Model Context Prot
ocol (MCP) servers and clients, simplifying the creation of LLM-integrated a
pplications with features like deployment, authentication, and dynamic tool
generation.
- Code Snippets: 347
- Source Reputation: High
- Benchmark Score: 71.71
- Versions: v3.0.0b1, v2.14.4, v3.0.0b2, v2.14.5, v3.0.0rc2, v3.0.1, v3.0.2,
v3.1.0
----------
- Title: FastMCP
- Context7-compatible library ID: /llmstxt/gofastmcp_llms_txt
- Description: FastMCP is a fast, Pythonic framework for building and deploy
ing Multi-modal Conversational Platform (MCP) servers and clients, offering
programmatic interfaces and extensive integrations for LLMs and various serv
ices.
- Code Snippets: 4366
- Source Reputation: High
- Benchmark Score: 76.22
----------
- Title: FastMCP
- Context7-compatible library ID: /llmstxt/gofastmcp_llms-full_txt
- Description: FastMCP
### Fetch All Tools Automatically (JavaScript)
```

```
Source: https://gofastmcp.com/servers/pagination

This snippet demonstrates how to fetch all available tools using the Gofastm
cp client. It automatically handles pagination to retrieve all 200 tools. Th
e output includes the total number of tools fetched.

```javascript
import { Client } from "fastmcp"

const client = new Client()

// Returns all 200 tools, fetching pages automatically
const tools = await client.list_tools()
print(f"Total tools: {le
```

Other Transports

There are other transports but we won't demo them here as they are not as widely used as STDIO or HTTP. These two are enough for most use cases of MCP servers.

SSE Transport

The Server-Sent Events transport enables HTTP communication between MCP servers and clients using an asymmetric channel. It uses SSE for server-to-client streaming and HTTP POST for client-to-server messages. It was replaced by the Streamable HTTP transport to provide bidirectional streaming capabilities and improved real-time communication efficiency.

Read more here.

In-Memory Transport

In-memory transport connects a client directly to a FastMCP server instance within the same Python process. This eliminates network overhead by enabling direct function calls between client and server components.

Read more here.

Referenced Sources

FastMCP
Context7

Authors
```

Joshua Zhou | Data Scientist @ IBM
Joseph Santarcangelo | Data Scientist @ IBM

Change Log

▶ Click here for the changelog

Copyright © IBM Corporation. All rights reserved.

In [ ]:

```
```